# LAPORAN HASIL PRAKTIKUM PRAKTIKUM PEMOGRAMAN BERBASIS OBJEK
# JOBSHEET IX
# KELAS ABSTRAK DAN INTERFACE

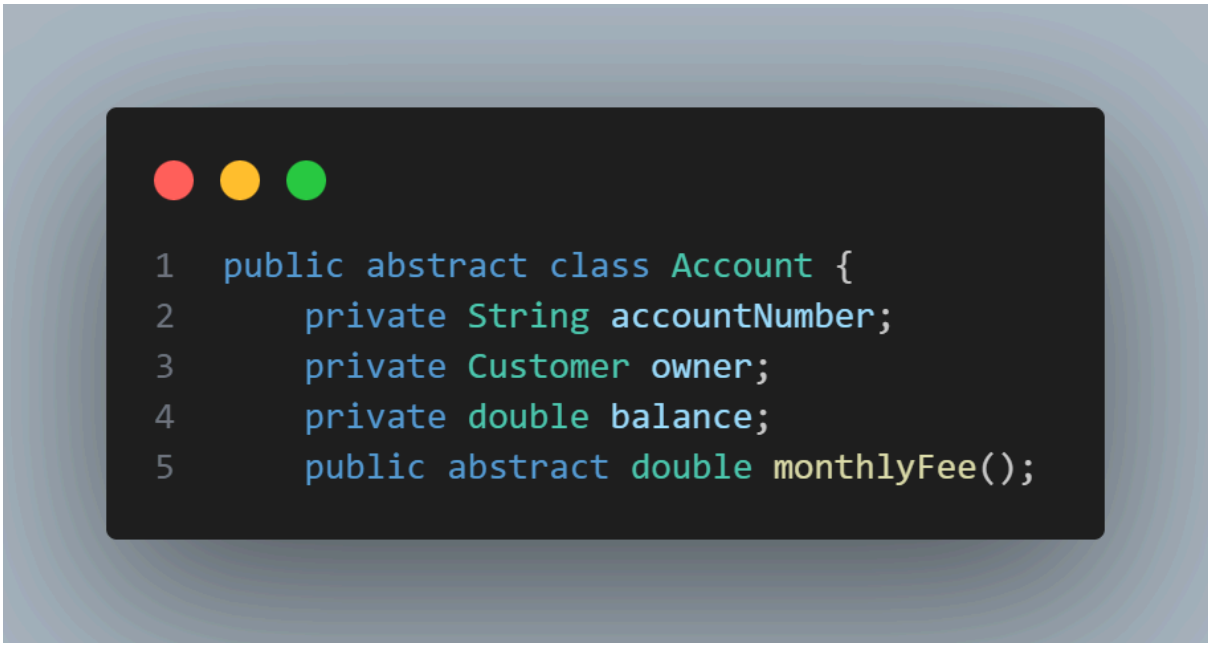



Disusun oleh: Muhammad Toriq Januarsyah
Kelas : TI-2B
NIM : 254107020233
Tanggal : 24 September 2026

## PROGRAM STUDI D-IV TEKNIK INFORMATIKA
## JURUSAN TEKNOLOGI INFORMASI
## POLITEKNIK NEGERI MALANG
## 2026

1. **Langkah Kerja - Account menjadi kelas Abstrak**

**1.1 mengubah class Account menjadi abstrak kelas dan menambahkan method monthlyFee()**

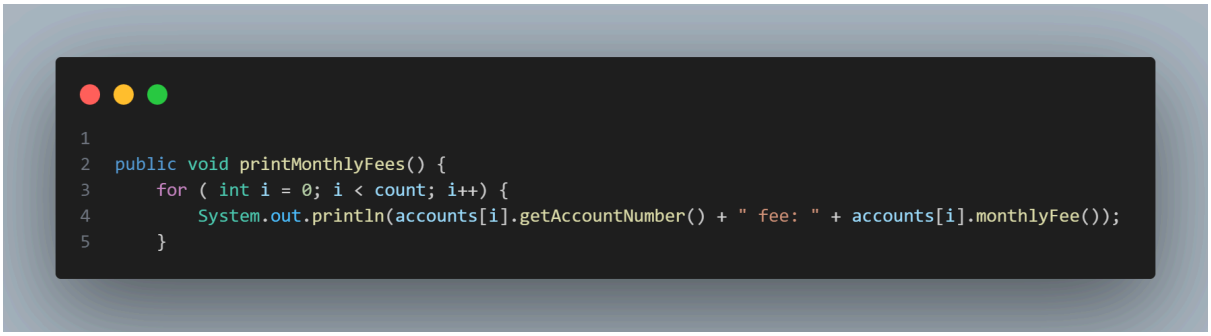


```
1  public abstract class Account {
2      private String accountNumber;
3      private Customer owner;
4      private double balance;
5      public abstract double monthlyFee();
```

*Gambar 1.1: Account. java sebagai abstrak class*

Kode ini menunjukkan pengubahan kelas Account menjadi kelas abstrak dengan menambahkan kata kunci abstract sebelum class Account. Di dalamnya dideklarasikan method abstrak public abstract double monthlyFee();. Method abstrak menetapkan contract atau janji bahwa setiap subclass turunan dari Account wajib mengimplementasikan method tersebut untuk menentukan besarnya biaya bulanan masing-masing.

**1.2 Tambahkan method printMonthlyFees() di Bank.java**



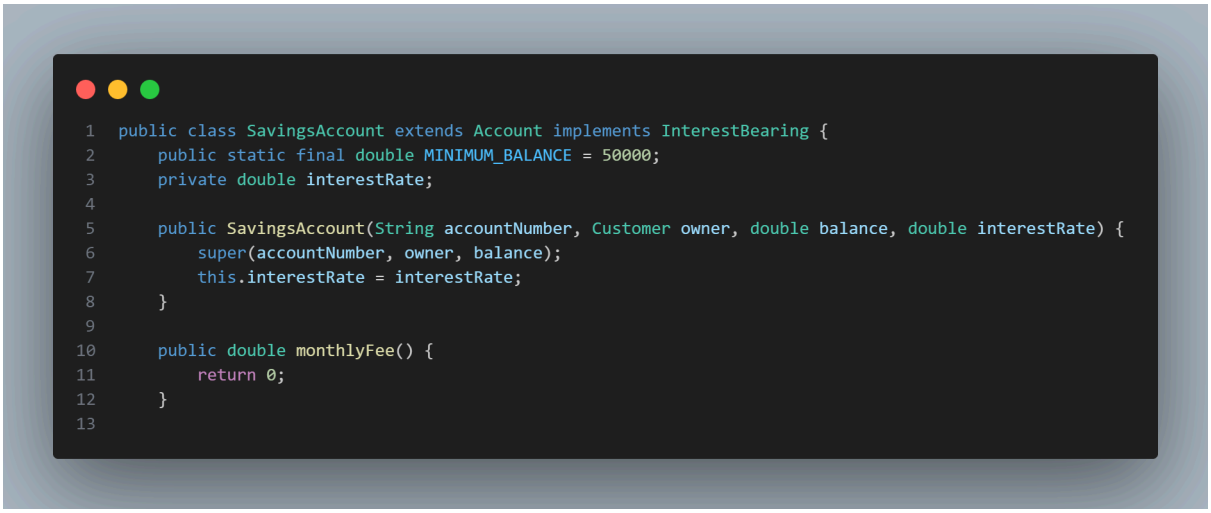
```
1
2  public void printMonthlyFees() {
3      for ( int i = 0; i < count; i++) {
4          System.out.println(accounts[i].getAccountNumber() + " fee: " + accounts[i].monthlyFee());
5      }
```

*gambar 1.2 :* Bank.java menambah method printMonthlyFees()

Method printMonthlyFees() ditambahkan pada kelas Bank untuk menampilkan rincian biaya bulanan dari seluruh rekening yang terdaftar. Method ini melakukan perulangan (looping) for sebanyak jumlah rekening yang ada (count). Di setiap perulangannya, program mengambil nomor rekening melalui getAccountNumber() dan memanggil method

monthlyFee() dari objek rekening yang tersimpan dalam array accounts untuk dicetak ke layar.

### 1.3 mengisi method monthlyFee() di savingAccount.java



```java
public class SavingsAccount extends Account implements InterestBearing {
    public static final double MINIMUM_BALANCE = 50000;
    private double interestRate;

    public SavingsAccount(String accountNumber, Customer owner, double balance, double interestRate) {
        super(accountNumber, owner, balance);
        this.interestRate = interestRate;
    }

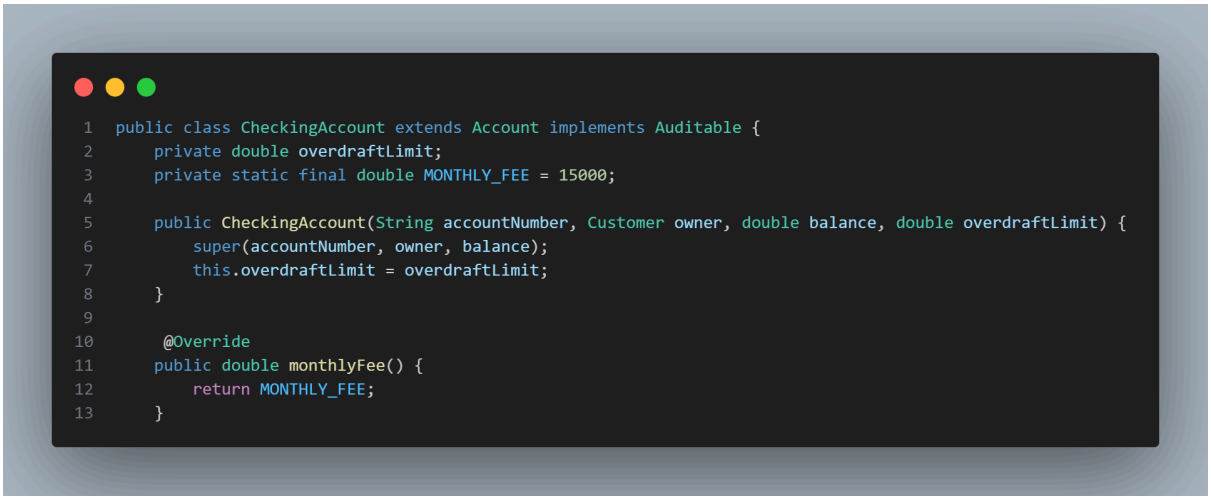
    public double monthlyFee() {
        return 0;
    }

```

*Gambar 1.3: SavingsAccount.java mengisi monthlyFee*

Pada kode ini, SavingsAccount merealisasikan method abstrak monthlyFee() dari superclass Account. Karena rekening tabungan (SavingsAccount) bebas biaya bulanan, method ini mengembalikan nilai 0. Pengisian method ini membebaskan SavingsAccount dari error kompilasi akibat mewarisi kelas abstrak.

### 1.4 Mengisi method monthlyFee() di checkingAccount.java



```java
public class CheckingAccount extends Account implements Auditable {
    private double overdraftLimit;
    private static final double MONTHLY_FEE = 15000;

    public CheckingAccount(String accountNumber, Customer owner, double balance, double overdraftLimit) {
        super(accountNumber, owner, balance);
        this.overdraftLimit = overdraftLimit;
    }

     @Override
    public double monthlyFee() {
        return MONTHLY_FEE;
    }
```

Gambar 1.4 *CheckingAccount.java mengisi monthlyFee*

Pada CheckingAccount, diumumkan konstanta MONTHLY_FEE sebesar 15000. Method abstrak monthlyFee() diimplementasikan dengan mengembalikan nilai konstanta MONTHLY_FEE tersebut, yang membebankan biaya administrasi bulanan tetap sebesar Rp15.000 untuk setiap rekening giro.

### 1.5 Perbarui Main.java

```java
public class Main {
    public static void main(String[] args) {
        Customer customer1 = new Customer("Nadia", "0812-0000-0001");
        SavingsAccount acc1 = new SavingsAccount("A001", customer1, 500000, 0.01);
        acc1.withdraw(150000);

        Customer customer2 = new Customer("Sari", "0812-0000-0002");
        CheckingAccount acc2 = new CheckingAccount("A002", customer2, 200000, 50000);

        Customer customer3 = new Customer("Rian", "0812-0000-0003");
        SavingsAccount savings = new SavingsAccount("A003", customer3, 100000, 0.02);
        savings.withdraw(70000);

        Customer customer4 = new Customer("Dewi", "0812-0000-0004");
        CheckingAccount checking = new CheckingAccount("A004", customer4, 100000, 200000);
        checking.withdraw(250000);

        Bank bank = new Bank(10);
        bank.addAccount(acc1);
        bank.addAccount(acc2);
        bank.addAccount(savings);
        bank.addAccount(checking);

        bank.printAllAccounts();
        bank.printMonthlyFees();
    }
}
```

*Gambar 1.5: Main.java memanggil printMonthlyFees*

Pada Main.java, beberapa objek SavingsAccount dan CheckingAccount dibuat dan dimasukkan ke dalam objek Bank. Kemudian dipanggil method bank.printAllAccounts() untuk mencetak data akun dan bank.printMonthlyFees() untuk mencetak riwayat/rincian biaya bulanan setiap rekening yang telah berhasil mengimplementasikan logika monthlyFee() masing-masing.

- **Checkpoint1:**

```
A001 - Nadia - balance: 350000.0
Account type: Saving, Interest rate: 0.01
A002 - Sari - balance: 200000.0
Account type: Checking, overdraft limit: 50000.0
A003 - Rian - balance: 100000.0
Account type: Saving, Interest rate: 0.02
A004 - Dewi - balance: -150000.0
Account type: Checking, overdraft limit: 200000.0
A001 fee: 0.0
A002 fee: 15000.0
A003 fee: 0.0
A004 fee: 15000.0
```

*Gambar 1.6: Output Main.java*

Delapan baris pertama merupakan hasil keluaran dari pemanggilan method bank.printAllAccounts():

- Rekening A001 (Nadia) bernilai saldo 350000.0 setelah dilakukan penarikan sebesar 150000 dari saldo awal 500000.
- Rekening A002 (Sari) menampilkan saldo awal 200000.0 dan batas overdraft 50000.0.
- Rekening A003 (Rian) tetap bernilai saldo 100000.0 karena transaksi penarikan sebesar 70000 ditolak akibat aturan saldo minimum.
- Rekening A004 (Dewi) bernilai saldo -150000.0 karena melakukan penarikan sebesar 250000 dari saldo awal 100000 menggunakan fasilitas overdraft.

Empat baris terakhir merupakan hasil keluaran dari pemanggilan method bank.printMonthlyFees():

- Rekening tabungan (A001 dan A003) menghasilkan biaya sebesar 0.0 sesuai implementasi method monthlyFee() pada SavingsAccount.
- Rekening giro (A002 dan A004) menghasilkan biaya tetap sebesar 15000.0 sesuai implementasi method monthlyFee() pada CheckingAccount.

## 2. Langkah kerja - InterestBearing, Interface untuk Rekening Berbunga

### 2.1 Buat berkas baru InterestBearing.java:

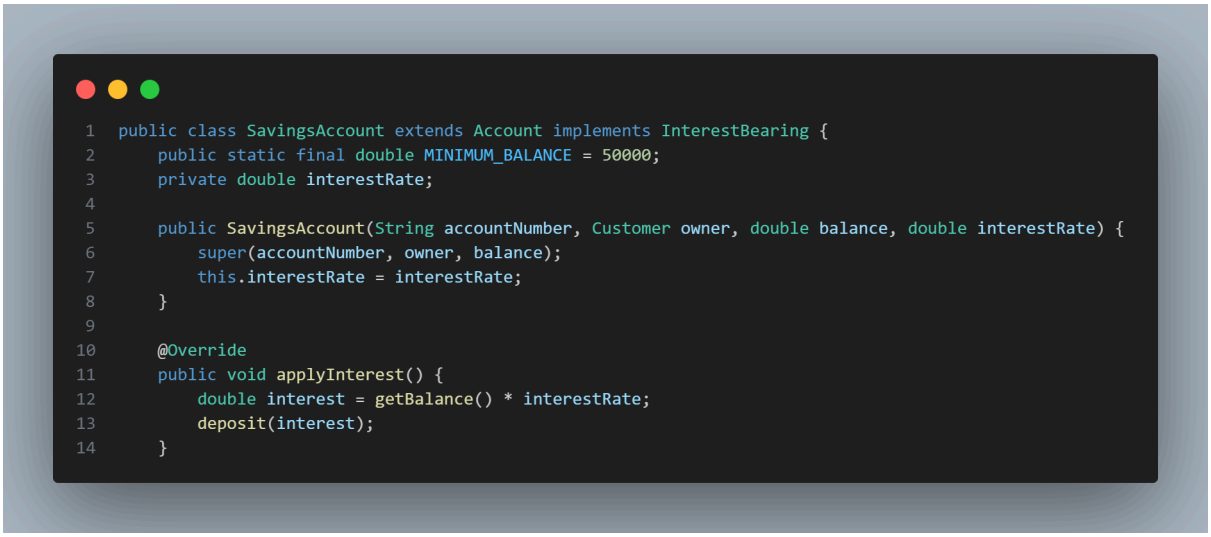
```java
package jobsheet9;

public interface InterestBearing {
    void applyInterest();
}
```

*Gambar 2.1: InterestBearing.java dengan method applyInterest();.*

Kode ini mendefinisikan interface baru bernama InterestBearing. Di dalamnya terdapat deklarasi method void applyInterest();. Secara otomatis, method di dalam interface berstatus public dan abstract, yang menetapkan bahwa kelas mana pun yang menerapkan interface ini wajib menyiagakan method applyInterest().

### 2.2 SavingsAccount.java. Tambahkan implements InterestBearing pada deklarasi kelas, lalu isi applyInterest()



```java
public class SavingsAccount extends Account implements InterestBearing {
    public static final double MINIMUM_BALANCE = 50000;
    private double interestRate;

    public SavingsAccount(String accountNumber, Customer owner, double balance, double interestRate) {
        super(accountNumber, owner, balance);
        this.interestRate = interestRate;
    }

    @Override
    public void applyInterest() {
        double interest = getBalance() * interestRate;
        deposit(interest);
    }
```

*Gambar 2.2: SavingsAccount.java meng-implement InterestBearing*

Kode ini menunjukkan penggunaan kata kunci implements InterestBearing pada deklarasi kelas SavingsAccount. Logika method applyInterest() diisi dengan menghitung besar bunga (saldo * interestRate) lalu menambahkan nominal bunga tersebut ke saldo akun melalui panggilan deposit(interest).

**2.3 pengujian di akhir Main.java. Rekening tabungan dipegang lewat variabel bertipe InterestBearing, sehingga pemanggil hanya mengenal kontraknya**

```java
package jobsheet9;


public class Main {
    public static void main(String[] args) {
        Customer customer1 = new Customer("Nadia", "0812-0000-0001");
        SavingsAccount acc1 = new SavingsAccount("A001", customer1, 500000, 0.01);
        acc1.withdraw(150000);

        Customer customer2 = new Customer("Sari", "0812-0000-0002");
        CheckingAccount acc2 = new CheckingAccount("A002", customer2, 200000, 50000);

        Customer customer3 = new Customer("Rian", "0812-0000-0003");
        SavingsAccount savings = new SavingsAccount("A003", customer3, 100000, 0.02);
        savings.withdraw(70000);

        Customer customer4 = new Customer("Dewi", "0812-0000-0004");
        CheckingAccount checking = new CheckingAccount("A004", customer4, 100000, 200000);
        checking.withdraw(250000);

        Bank bank = new Bank(10);
        bank.addAccount(acc1);
        bank.addAccount(acc2);
        bank.addAccount(savings);
        bank.addAccount(checking);

        bank.printAllAccounts();
        bank.printMonthlyFees();

        System.out.println("Before interest: " + savings.getBalance());
        savings.applyInterest();
        System.out.println("After interest: " + savings.getBalance());
    }
}

```

*Gamabr 2.3: Main.java menguji applyInterest*

Kode ini menguji penerapan interface dengan menggunakan polimorfisme interface. Objek savings dirujuk oleh variabel tipe interface InterestBearing earner = savings;. Melalui variabel earner, program menjalankan applyInterest(), yang secara transparan menambah saldo bunga pada objek savings tersebut. Ini membuktikan bahwa pemanggil cukup berinteraksi dengan tipe interface tanpa perlu tahu seluruh detail internal kelas implementasinya.

- **Checkpoint2:**

```
A001 - Nadia - balance: 350000.0
Account type: Saving, Interest rate: 0.01
A002 - Sari - balance: 200000.0
Account type: Checking, overdraft limit: 50000.0
A003 - Rian - balance: 100000.0
Account type: Saving, Interest rate: 0.02
A004 - Dewi - balance: -150000.0
Account type: Checking, overdraft limit: 200000.0
A001 fee: 0.0
A002 fee: 15000.0
A003 fee: 0.0
A004 fee: 15000.0
Before interest: 100000.0
After interest: 102000.0
```

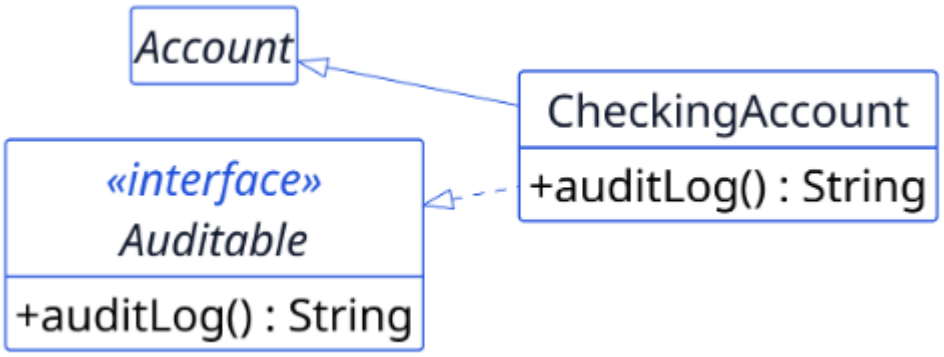
*Gambar 2.4: Output Main.java*

Dua baris terakhir (Before interest & After interest) merupakan hasil pengujian interface InterestBearing pada Langkah 2:

- Before interest: 100000.0 menunjukkan saldo awal rekening A003 (Rian).
- After interest: 102000.0 menunjukkan saldo setelah pemanggilan earner.applyInterest(). Suku bunga sebesar 2% (0.02) dikalikan saldo 100000.0 menghasilkan bunga Rp2.000, yang otomatis ditambahkan ke dalam saldo.

**3. Tugas Mandiri.**

1. Bank memerlukan jejak audit untuk rekening yang saldonya bisa negatif (rekening dengan overdraft). Tambahkan interface Auditable, lalu terapkan pada CheckingAccount. Diagram berikut hanya sketsa method yang perlu diisi, BUKAN kode jadi, isi auditLog() diserahkan sepenuhnya padamu:




Buktikan lewat Main.java: panggil auditLog() pada kedua CheckingAccount yang sudah ada, lalu cetak hasilnya. Hasilmu benar apabila:

- Auditable adalah interface dengan satu method auditLog();
- CheckingAccount menyatakan implements Auditable, sedangkan SavingsAccount tidak;
- teks yang dicetak memuat nomor rekening dan saldonya.

- **Class interface Auditable.java**



```
1  package jobsheet9;
2
3  public interface Auditable {
4      String auditLog();
5  }
6
```

*Gambar 3.1: Class Auditable dengan method auditLog()*

*Interface Auditable menyediakan kontrak method auditLog() yang mengembalikan String berisi ringkasan informasi audit rekening.*

- **Class CheckingAccount implements auditable**

```java
package jobsheet9;

public class CheckingAccount extends Account implements Auditable {
    private double overdraftLimit;
    private static final double MONTHLY_FEE = 15000;

    public CheckingAccount(String accountNumber, Customer owner, double balance, double overdraftLimit) {
        super(accountNumber, owner, balance);
        this.overdraftLimit = overdraftLimit;
    }

     @Override
    public double monthlyFee() {
        return MONTHLY_FEE;
    }

    public double getOverdraftLimit() {
        return overdraftLimit;
    }

    @Override
    protected boolean canWithdraw(double amount){
        return amount > 0 && amount <= getBalance() + overdraftLimit;
    }

    @Override
    public void printInfo(){
        super.printInfo();
        System.out.println("Account type: Checking, overdraft limit: " + overdraftLimit);
    }

    public void printAccountType() {
        System.out.println("Account type: Checking, overdraft limit: " + overdraftLimit);
    }

   @Override
    public String auditLog() {
        return "Audit Log - Account: " + getAccountNumber() + ", Balance: " + getBalance();
    }
}
```

*Gambar 3.2: Class CheckingAccount implements Auditable dan ovveride auditLog*

CheckingAccount ditambahkan klausa implements Auditable dan mengimplementasikan method auditLog(). Method ini mengembalikan string format yang mencantumkan nomor rekening serta saldo terkini untuk kebutuhan pengawasan log audit.

- **Pengujian di Main.java**

```java
package jobsheet9;


public class Main {
    public static void main(String[] args) {
        Customer customer1 = new Customer("Nadia", "0812-0000-0001");
        SavingsAccount acc1 = new SavingsAccount("A001", customer1, 500000, 0.01);
        acc1.withdraw(150000);

        Customer customer2 = new Customer("Sari", "0812-0000-0002");
        CheckingAccount acc2 = new CheckingAccount("A002", customer2, 200000, 50000);

        Customer customer3 = new Customer("Rian", "0812-0000-0003");
        SavingsAccount savings = new SavingsAccount("A003", customer3, 100000, 0.02);
        savings.withdraw(70000);

        Customer customer4 = new Customer("Dewi", "0812-0000-0004");
        CheckingAccount checking = new CheckingAccount("A004", customer4, 100000, 200000);
        checking.withdraw(250000);

        Bank bank = new Bank(10);
        bank.addAccount(acc1);
        bank.addAccount(acc2);
        bank.addAccount(savings);
        bank.addAccount(checking);

        bank.printAllAccounts();
        bank.printMonthlyFees();

        System.out.println("Before interest: " + savings.getBalance());
        savings.applyInterest();
        System.out.println("After interest: " + savings.getBalance());

        System.out.println(acc2.auditLog());
        System.out.println(checking.auditLog());
    }
}

```

*Gambar 3.3: memanggil auditLog()*

dua baris kode terkahir memanggil method auditLog() dari kedua objek CheckingAccount (acc2 dan checking) lalu mencetak hasilnya ke konsol untuk mengonfirmasi bahwa interface Auditable telah bekerja dengan benar.

- **Output**

```
A001 - Nadia - balance: 350000.0
Account type: Saving, Interest rate: 0.01
A002 - Sari - balance: 200000.0
Account type: Checking, overdraft limit: 50000.0
A003 - Rian - balance: 100000.0
Account type: Saving, Interest rate: 0.02
A004 - Dewi - balance: -150000.0
Account type: Checking, overdraft limit: 200000.0
A001 fee: 0.0
A002 fee: 15000.0
A003 fee: 0.0
A004 fee: 15000.0
Before interest: 100000.0
After interest: 102000.0
Audit Log - Account: A002, Balance: 200000.0
Audit Log - Account: A004, Balance: -150000.0
```

Gambar 3.4: output Main.java

Dua baris terakhir (Audit Log ...) merupakan hasil eksekusi penggunaan interface Auditable pada rekening giro (CheckingAccount):

- Baris pertama mencetak log audit untuk rekening A002 (Sari) dengan saldo sebesar 200000.0.
- Baris kedua mencetak log audit untuk rekening A004 (Dewi) dengan saldo sebesar -150000.0.
- Keluaran ini membuktikan bahwa method auditLog() pada interface Auditable yang diimplementasikan oleh CheckingAccount telah berhasil menampilkan nomor rekening beserta saldonya sesuai dengan ketentuan tugas.

2. Jawab secara singkat (1-2 kalimat untuk masing-masing pertanyaan):
   - (a) Mengapa applyInterest() dijadikan interface InterestBearing, bukan method abstrak di Account?

     Karena tidak semua jenis rekening memiliki kemampuan untuk menghasilkan/menerima bunga (contohnya CheckingAccount tidak berbunga). Jika ditaruh di Account sebagai method abstrak, maka seluruh subclass dipaksa untuk mengimplementasikannya, padahal fitur tersebut hanya relevan untuk jenis rekening tertentu seperti SavingsAccount.

   - (b) SavingsAccount memakai extends Account dan implements InterestBearing. Apa beda makna kedua kata kunci itu?

     extends Account berarti SavingsAccount me-wariskan struktur utama, data, dan perilaku dasar dari kelas induk Account (hubungan hirarki is-a). Sedangkan implements InterestBearing berarti SavingsAccount mengadopsi/memenuhi kontrak kemampuan tambahan dari interface InterestBearing (hubungan kemampuan can-do).

3. **Kesimpulan**

   Berdasarkan praktikum dan penyelesaian tugas mandiri yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

   - Penggunaan Kelas Abstrak (Abstract Class): Kelas abstrak digunakan sebagai fondasi dasar (template) yang tidak dapat diinstansiasi secara langsung menggunakan kata kunci new. Pendekatan ini memastikan bahwa kelas induk seperti Account menetapkan method abstrak (monthlyFee()) yang wajib diimplementasikan oleh setiap subclass (SavingsAccount dan CheckingAccount) sesuai dengan karakteristik bisnis masing-masing.
   - Penggunaan Interface: Interface berfungsi sebagai kontrak kemampuan spesifik yang bersifat opsional dan dapat diterapkan pada kelas mana pun. Dengan memisahkan fitur seperti InterestBearing (untuk perhitungan bunga) dan Auditable (untuk pencatatan log audit), kelas hanya akan mengambil kemampuan yang benar-benar diperlukannya tanpa membebani kelas lain yang tidak membutuhkan fitur tersebut.
   - Perbedaan extends dan implements: Kata kunci extends digunakan untuk mewarisi struktur dan perilaku dasar dari satu kelas induk (single inheritance), sedangkan implements digunakan untuk mengadopsi satu atau beberapa kontrak kemampuan dari interface (multiple interface implementation).

4. **Lampiran**

   https://github.com/ToriqJanuarsyah/Praktikum_PBO.git